# Table of Contents


📐 Architecture Technique - InSport ..... 5

🏢 Backend (Spring Boot) ..... 5

🎮 Moteur de Jeu & Logique Métier ..... 6

🔌 API REST ..... 6

Structure des packages ..... 6

Base de données ..... 6

Sécurité ..... 6

🎨 Frontend (React) ..... 6

Fonctionnalités Clés ..... 6

Communication avec le backend ..... 7

📊 Diagrammes UML ..... 8

Diagramme de Classes (Modèle de Données) ..... 8

Diagramme de Séquence (Conquête d'un Point) ..... 10

🔗 Configuration Front-End ↔ Back-End ↔ Base de Données ..... 10

📋 Vue d'ensemble de l'architecture ..... 10

1️⃣ Configuration de la Base de Données ..... 11

Fichier: `app/src/main/resources/application.properties` ..... 11

Création de la BD ..... 11

2️⃣ Entités JPA (Modèle ↔ BD) ..... 11

Exemple: Équipe ..... 11

3️⃣ Repositories Spring Data JPA ..... 12

4️⃣ Contrôleurs REST (API) ..... 12

Endpoints disponibles: ..... 13

5️⃣ Configuration CORS (Front-End → Back-End) ..... 13

6️⃣ Configuration Vite (Proxy) ..... 13

7️⃣ Service API React ..... 14

Utilisation dans les composants: ..... 14

✅ Checklist de démarrage ..... 14

🧪 Page de Test ..... 15

Accès: `http://localhost:5173/` (après intégration dans App.jsx) ..... 15

🔧 Dépannage ..... 15

Erreur 1: “CORS error” ..... 15
Erreur 2: “Base de données introuvable” ..... 15
Erreur 3: “Cannot resolve symbol” ..... 15
Erreur 4: “Cannot GET /api/…” ..... 15
📚 Ressources ..... 15
Base de données SQLite - SportsIn ..... 16
Création de la base de données ..... 16
Structure de la base de données ..... 16
Tables principales ..... 16
Relations ..... 17
Utilisation ..... 17
Accéder à la base de données ..... 17
Commandes SQLite utiles ..... 17
Exemples de requêtes ..... 18
Fichiers ..... 18
Feature 6 : Team Progression & Unlockable Perks ..... 18
1. Vue d’ensemble ..... 18
Objectifs ..... 18
2. Architecture & Design Patterns ..... 19
Strategy Pattern ..... 19
Registry Pattern ..... 19
Chain of Responsibility (Modifier Chain) ..... 19
Principes SOLID ..... 19
3. Systeme de niveaux ..... 19
Gain d’XP ..... 20
4. Perks disponibles ..... 20
5. Endpoints API REST ..... 20
`GET /api/teams/{teamId}/progression` ..... 20
`GET /api/perks` ..... 21
`POST /api/teams/{teamId}/perks/activate` ..... 21
`GET /api/teams/{teamId}/perks/active` ..... 21
6. Fichiers crees (18 fichiers) ..... 21
Modeles ..... 21
Strategies (effets) ..... 21

Services ..... 22
Controleur ..... 22
Repositories ..... 22
7. Fichiers modifies (6 fichiers) ..... 22
8. Tests (34 tests Feature 6) ..... 23
LevelThresholdTest — 14 tests ..... 23
ShieldEffectTest — 9 tests ..... 23
InfluenceCalculatorTest — 4 tests ..... 23
TeamProgressionServiceTest — 7 tests ..... 23
Resultat global ..... 23
9. Schema de base de donnees ..... 23
Table `perk_definition` ..... 23
Table `active_perk` ..... 24
10. Diagramme de flux ..... 24
11. Comment ajouter un nouveau perk ..... 25
12. Branche Git ..... 25
Feature 3 : Calcul Automatique de Contrôle de Points et de Zones ..... 26
1. Objectifs ..... 26
2. Fonctionnement Technique ..... 26
A. Génération Automatique des Zones (`ZoneGeneratorService`) ..... 26
B. Mécanique de Conquête (`TerritoryService`) ..... 26
3. Architecture et Composants Clés ..... 26
4. Exemple de Scénario (Logique) ..... 27
5. Tests ..... 27
Feature 4 : Routes sportives & bonus de combo ..... 27
Objectif ..... 27
Statut ..... 27
Architecture & Implémentation ..... 27
1. Modèle de Données ..... 27
2. Services Métier ..... 28
3. API & Configuration ..... 28
4. Tests ..... 28
Flux Fonctionnel ..... 28
Feature 5 : Systeme de Missions Dynamiques ..... 29

1. Vue d’ensemble ..... 29
Objectifs ..... 29
2. Architecture & Design Patterns ..... 29
Strategy Pattern ..... 29
Repository Pattern ..... 29
DTO Pattern ..... 29
Payload Pattern ..... 29
Scheduled Tasks ..... 30
3. Types de Missions ..... 30
Regles de generation (R1, R2, R3) ..... 30
4. Scheduler ..... 30
5. Endpoints API REST ..... 31
`GET /api/teams/{teamId}/missions` ..... 31
`GET /api/missions/{missionId}` ..... 31
`POST /api/teams/{teamId}/missions/generate` ..... 31
`POST /api/missions/{missionId}/refresh` ..... 31
6. Fichiers crees (15 fichiers) ..... 31
Modeles ..... 31
DTOs ..... 32
Services ..... 32
Scheduler ..... 32
Controleur ..... 32
Repository ..... 32
Tests ..... 32
Frontend ..... 32
7. Fichiers modifies ..... 33
8. Tests ..... 33
MissionServiceTest — 9 tests ..... 33
9. Schema de base de donnees ..... 33
Table `mission` ..... 33
Index de performance ..... 34
10. Diagramme de flux ..... 34
11. Integration Frontend ..... 35
Marqueurs de mission ..... 35

Labels francais ..... 35
Popup de mission ..... 36
12. Branche Git ..... 36
🎮 Mécaniques de Jeu - InSport ..... 36
🗺️ Territoire & Points d'Intérêt ..... 36
📍 Points d'Intérêt (POI) ..... 36
🏙️ Zones ..... 36
🛣️ Routes Sportives (Innovation) ..... 36
🏆 Sessions Sportives & Conquête ..... 37
🎯 Missions Dynamiques ..... 37
📊 Diagramme des Cas d'Utilisation ..... 37
📅 Plan de Livraisons - InSport ..... 38
🎁 Livraison 1 : Socle du Jeu & Règles Sportives ..... 38
✨ Feature 1 : Moteur de règles de victoire multi-sports configurable ..... 38
✨ Feature 2 : Gestion des sessions de défi & Validation ..... 38
🎁 Livraison 2 : Territoire, Zones & Routes (Innovation) ..... 39
✨ Feature 3 : Calcul de contrôle de Points & Zones ..... 39
✨ Feature 4 : Routes Sportives & Bonus de Combo (Innovation Majeure) ..... 39
🎁 Livraison 3 : Engagement & Personnalisation ..... 39
✨ Feature 5 : Missions Dynamiques ..... 39
✨ Feature 6 : Progression d'Équipe & Perks ..... 40
📝 Résumé ..... 40

# □ Architecture Technique - InSport

Ce document décrit l'architecture technique de l'application InSport, décomposée en deux parties principales : le backend et le frontend.

## □ Backend (Spring Boot)

Le backend est le cœur du projet. Il est totalement autonome, solide et cohérent. Il gère toute la logique métier, les règles du jeu et l'API REST.

### □ Moteur de Jeu & Logique Métier

Le backend gère : * **Modèles de données** : Équipes, Sports, Sessions, Résultats, Territoires (Points, Zones, Routes). * **Moteur de règles multi-sports** : Applique les règles spécifiques à chaque sport pour valider les sessions. * **Calcul d'influence** : Algorithmes pour déterminer le contrôle des points et des zones. * **Algorithmes de graphes** : Gestion des routes sportives et des bonus associés. * **Missions dynamiques** : Génération et suivi des missions. * **Progression d'équipe** : Système de niveaux et de récompenses.

### □ API REST

L'API est indépendante du front-end et expose : * La liste des sports et leurs règles. * Les points, zones et routes. * Les sessions et résultats. * Les missions. * Les bonus et perks. * Des endpoints de validation et de résolution de conflits.

### Structure des packages

Le code source est organisé de la manière suivante : - `com.example.sportsin.model` : Contient les entités JPA (User, Team, Event, Point, Zone, Route). - `com.example.sportsin.repository` : Interfaces Spring Data JPA. - `com.example.sportsin.service` : Logique métier (GameEngine, TerritoryService, MissionService). - `com.example.sportsin.controller` : Contrôleurs REST. - `com.example.sportsin.config` : Configuration (Sécurité, Base de données).

### Base de données

L'application utilise une base de données **SQLite** pour la persistance des données. Le schéma est détaillé dans DATABASE.md.

### Sécurité

La sécurité est gérée par **Spring Security**. L'accès à l'API est protégé et nécessite une authentification (JWT).

## □ Frontend (React)

Le frontend est une application monopage (SPA) développée avec **React** et **Vite**. Il sert d'interface utilisateur pour interagir avec le jeu.

### Fonctionnalités Clés

- **Carte dynamique** : Affichage des points, zones et routes sur une carte interactive.
- **Interaction joueur** : Création d'équipes, lancement de sessions, soumission de résultats.
- **Tableau de bord** : Suivi des missions, des bonus et de la progression.

## Communication avec le backend

Le frontend communique avec le backend via des requêtes HTTP aux endpoints de l’API REST. Il est conçu pour être remplaçable par n’importe quel autre client (mobile, CLI, etc.), car toute la logique réside dans le backend.

## □ Diagrammes UML

### Diagramme de Classes (Modèle de Données)

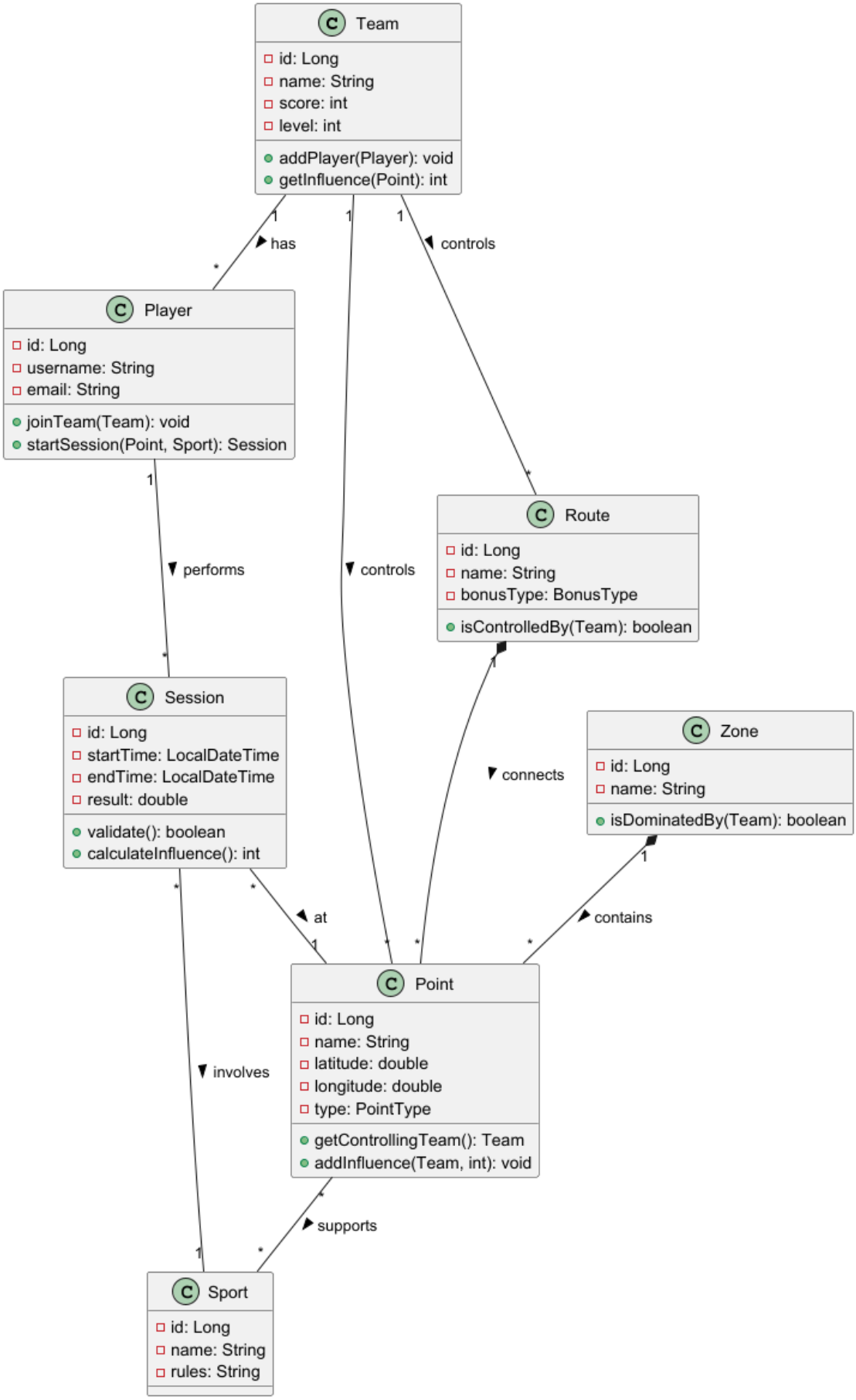
Team
id: Long
name: String
score: int
level: int
addPlayer(Player): void
getInfluence(Point): int
has
controls
Player
id: Long
username: String
email: String
joinTeam(Team): void
startSession(Point, Sport): Session
performs
controls
Route
id: Long
name: String
bonusType: BonusType
isControlledBy(Team): boolean
Session
id: Long
startTime: LocalDateTime
endTime: LocalDateTime
result: double
validate(): boolean
calculateInfluence(): int
connects
Zone
id: Long
name: String
isDominatedBy(Team): boolean
at
contains
involves
Point
id: Long
name: String
latitude: double
longitude: double
type: PointType
getControllingTeam(): Team
addInfluence(Team, int): void
supports
Sport
id: Long
name: String
rules: String

*Diagramme de Classes*

## Diagramme de Séquence (Conquête d'un Point)

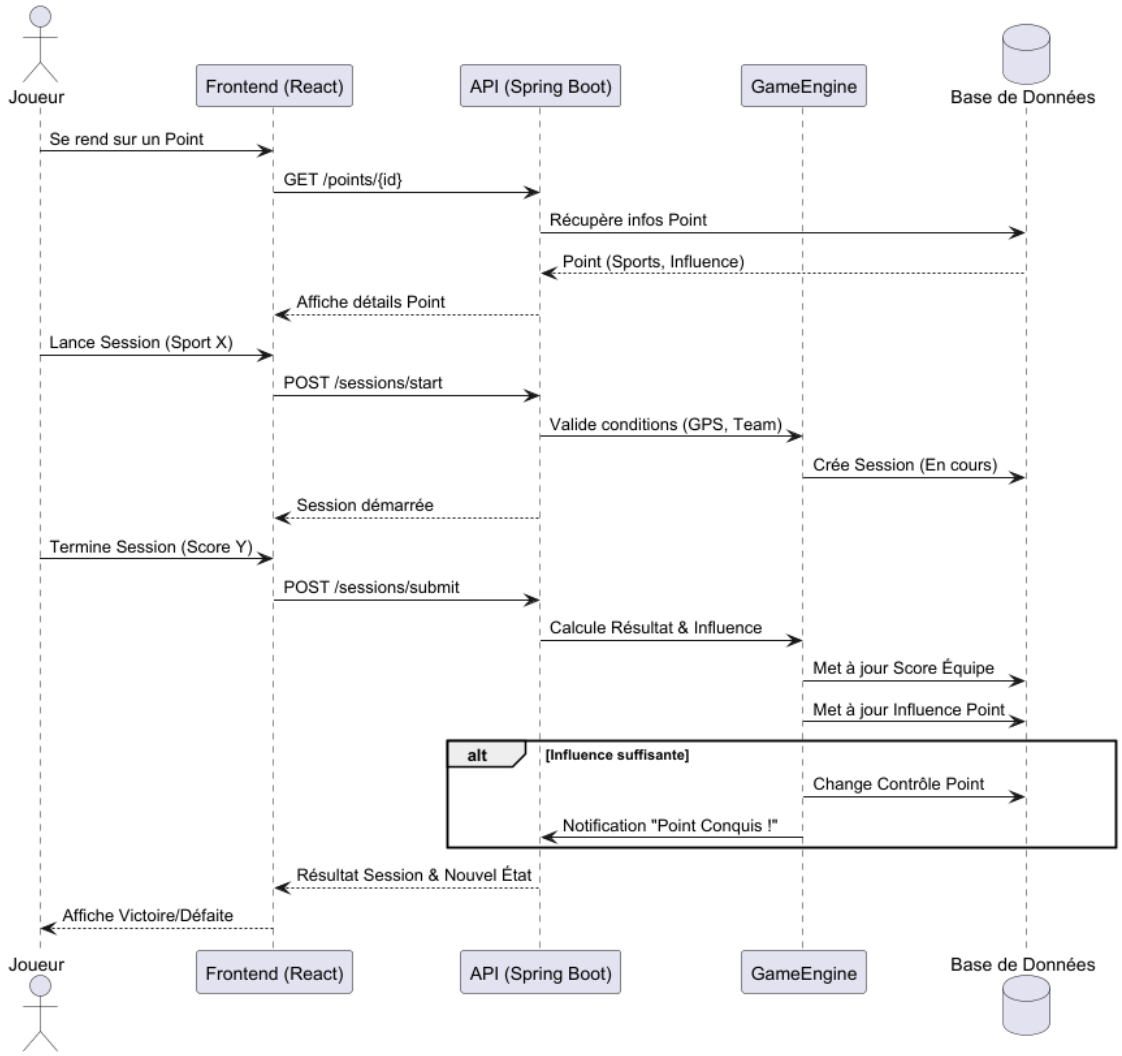


*Diagramme de Séquence*

# □ Configuration Front-End ↔ Back-End ↔ Base de Données

Ce document explique comment le projet **SportsIn** relie le front-end React/Vite, le back-end Spring Boot et la base de données SQLite.

## □ Vue d'ensemble de l'architecture

```
┌─────────────────────────────────────────────────────────┐
│  FRONTEND (React + Vite)                                │
│  Port: 5173                                             │
│  - App.jsx                                              │
```

```
│  - Pages (HomePage, MapPage, etc.)                    │
│  - api/api.js (service API)                           │
└──────────────────────────────────────────────────────┘
                        │ HTTP requests (fetch)
                        │ /api/equipes, /joueurs, etc.
                        ↓
┌──────────────────────────────────────────────────────┐
│  BACKEND (Spring Boot)                                │
│  Port: 8080                                           │
│  - Controllers (@RestController)                      │
│  - Services (@Service)                                │
│  - Repositories (JpaRepository)                       │
│  - Models (Entity @Entity)                            │
└──────────────────────────────────────────────────────┘
                        │ JPA/Hibernate
                        │ SQL queries
                        ↓
┌──────────────────────────────────────────────────────┐
│  DATABASE (SQLite)                                    │
│  File: sportsin.db                                    │
│  - Tables: equipe, joueur, arene, sport, session    │
│  - Relationships: FK, Indexes                         │
└──────────────────────────────────────────────────────┘
```

## 1□ Configuration de la Base de Données

### Fichier: `app/src/main/resources/application.properties`

```
# SQLite Configuration
spring.datasource.url=jdbc:sqlite:sportsin.db
spring.datasource.driver-class-name=org.sqlite.JDBC

# JPA/Hibernate
spring.jpa.database-platform=org.hibernate.community.dialect.SQLiteDialect
spring.jpa.hibernate.ddl-auto=validate
```

### Création de la BD

```
./create_database.sh
```

Cela exécute le script SQL dans `app/src/main/resources/schema.sql` qui crée : - ✅ Table `equipe` - ✅Table `joueur` - ✅Table `arene` - ✅Table `sport` - ✅Table `session` - ✅ Table `metric_value` - ✅Tables de jointure (many-to-many)

## 2□ Entités JPA (Modèle ↔ BD)

Les classes `Entity` font le pont entre Java et la base de données :

### Exemple: Équipe

```
@Entity
@Table(name = "equipe")
```

```java
public class Equipe {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true)
    private String nom;

    @OneToMany(mappedBy = "equipe", cascade = CascadeType.ALL)
    private Set<Joueur> joueurs;
}
```

**Fichiers concernés:** - `app/src/main/java/org/SportsIn/model/Equipe.java` - `app/src/main/java/org/SportsIn/model/Joueur.java` - `app/src/main/java/org/SportsIn/model/Arene.java`

## 3□ Repositories Spring Data JPA

Les repositories permettent d'accéder à la BD sans écrire du SQL :

```java
@Repository
public interface EquipeRepository extends JpaRepository<Equipe, Long> {
    Optional<Equipe> findByNom(String nom);
}
```

**Méthodes disponibles automatiquement:** - `findAll()` - Récupère toutes les équipes - `findById(Long id)` - Récupère une équipe par ID - `save(Equipe equipe)` - Crée ou modifie une équipe - `delete(Equipe equipe)` - Supprime une équipe

**Fichiers:** - `app/src/main/java/org/SportsIn/repository/EquipeRepository.java` - `app/src/main/java/org/SportsIn/repository/JoueurRepository.java` - `app/src/main/java/org/SportsIn/repository/AreneRepository.java`

## 4□ Contrôleurs REST (API)

Les contrôleurs exposent des endpoints HTTP pour le front-end :

```java
@RestController
@RequestMapping("/api/equipes")
@CrossOrigin(origins = {"http://localhost:5173", "http://localhost:3000"})
public class EquipeController {

    @GetMapping
    public ResponseEntity<List<Equipe>> getAll() {
        return ResponseEntity.ok(equipeRepository.findAll());
    }

    @PostMapping
    public ResponseEntity<Equipe> create(@RequestBody Equipe equipe) {
        return ResponseEntity.status(HttpStatus.CREATED)
```

```
            .body(equipeRepository.save(equipe));
    }
}
```

### Endpoints disponibles:

| Méthode | URL | Description |
|---|---|---|
| GET | `/api/equipes` | Récupère toutes les équipes |
| GET | `/api/equipes/{id}` | Récupère une équipe |
| POST | `/api/equipes` | Crée une équipe |
| PUT | `/api/equipes/{id}` | Modifie une équipe |
| DELETE | `/api/equipes/{id}` | Supprime une équipe |

**Même pattern pour:** `/api/joueurs`, `/api/arenes`

**Fichiers:** - `app/src/main/java/org/SportsIn/controller/EquipeController.java` - `app/src/main/java/org/SportsIn/controller/JoueurController.java` - `app/src/main/java/org/SportsIn/controller/AreneController.java`

## 5□ Configuration CORS (Front-End → Back-End)

Le CORS (Cross-Origin Resource Sharing) permet au front-end d'accéder à l'API du back-end:

**Fichier:** `app/src/main/java/org/SportsIn/config/CorsConfig.java`

```
@Configuration
public class CorsConfig implements WebMvcConfigurer {
    @Override
    public void addCorsMappings(CorsRegistry registry) {
        registry.addMapping("/api/**")
                .allowedOrigins("http://localhost:5173", "http://localhost:30
00")
                .allowedMethods("GET", "POST", "PUT", "DELETE")
                .allowCredentials(true);
    }
}
```

## 6□ Configuration Vite (Proxy)

**Fichier:** `frontend/vite.config.js`

```
export default defineConfig({
  server: {
    port: 5173,
    proxy: {
      '/api': {
        target: 'http://localhost:8080',
        changeOrigin: true,
        rewrite: (path) => path.replace(/^\/api/, '/api')
```

```
      }
    }
  }
})
```

**Bénéfice:** Pendant le développement, les requêtes `/api/*` sont automatiquement redirigées vers `http://localhost:8080/api/*`

## 7□ Service API React

**Fichier:** `frontend/src/api/api.js`

```
const API_BASE_URL = '/api'; // Utilise le proxy Vite

export const equipeAPI = {
  getAll: async () => fetchAPI('/equipes'),
  create: async (data) => fetchAPI('/equipes', {
    method: 'POST',
    body: JSON.stringify(data)
  }),
};
```

### Utilisation dans les composants:

```
import { equipeAPI } from '../api/api';

export default function MyComponent() {
  const [equipes, setEquipes] = useState([]);

  useEffect(() => {
    equipeAPI.getAll()
      .then(data => setEquipes(data))
      .catch(error => console.error(error));
  }, []);

  return (
    <div>
      {equipes.map(equipe => (
        <div key={equipe.id}>{equipe.nom}</div>
      ))}
    </div>
  );
}
```

## ✅Checklist de démarrage

```
# 1. Créer la base de données
./create_database.sh

# 2. Démarrer le back-end Spring Boot
./gradlew bootRun
```

```
# ✅ Backend accessible à http://localhost:8080

# 3. Installer les dépendances front-end
cd frontend
npm install

# 4. Démarrer le front-end Vite
npm run dev
# ✅ Frontend accessible à http://localhost:5173
```

## 🧪 Page de Test

Une page de test est disponible pour vérifier que tout fonctionne :

**Fichier:** `frontend/src/pages/ApiTestPage.jsx`

Elle permet de : - ✅Récupérer toutes les équipes, joueurs et arènes - ✅Créer une nouvelle équipe - ✅Créer un nouveau joueur - ✅Supprimer des données

### Accès: `http://localhost:5173/` (après intégration dans App.jsx)

## 🔧 Dépannage

### Erreur 1: “CORS error”

**Solution:** Vérifier que `CorsConfig.java` est activé et que les ports sont corrects.

### Erreur 2: “Base de données introuvable”

**Solution:** Exécuter `./create_database.sh`

### Erreur 3: “Cannot resolve symbol”

**Solution:**

```
# Rebuilder le projet
./gradlew clean build
```

### Erreur 4: “Cannot GET /api/…”

**Solution:** S’assurer que le backend est lancé sur le port 8080

## 📚 Ressources

- Spring Data JPA Documentation
- Hibernate Documentation
- React Hooks Documentation
- SQLite Documentation

# Base de données SQLite - SportsIn

Ce document décrit la structure de la base de données SQLite du projet SportsIn.

## Création de la base de données

Pour créer la base de données SQLite, exécutez le script suivant :

```
./create_database.sh
```

Ce script crée un fichier `sportsin.db` dans le répertoire racine du projet.

## Structure de la base de données

### Tables principales

1. **EQUIPE** - Stocke les équipes
   - `id` (INTEGER, PK, AUTOINCREMENT)
   - `nom` (TEXT, UNIQUE, NOT NULL)
2. **JOUEUR** - Stocke les joueurs individuels
   - `id` (INTEGER, PK, AUTOINCREMENT)
   - `pseudo` (TEXT, UNIQUE, NOT NULL)
   - `equipe_id` (INTEGER, FK vers EQUIPE, nullable)
3. **ARENE** - Stocke les arènes (points sportifs sur la carte)
   - `id` (TEXT, PK)
   - `nom` (TEXT, NOT NULL)
   - `latitude` (REAL, NOT NULL)
   - `longitude` (REAL, NOT NULL)
   - `equipe_controle` (INTEGER, FK vers EQUIPE, nullable)
4. **ARENE_SPORT** - Table de jointure (ManyToMany)
   - `arene_id` (TEXT, FK vers ARENE, PK)
   - `sport_type` (TEXT, PK) - Valeurs: ‘FOOTBALL’, ‘MUSCULATION’, ‘BASKET’, ‘TENNIS’
5. **SPORT** - Stocke les sports disponibles
   - `id` (INTEGER, PK, AUTOINCREMENT)
   - `code` (TEXT, UNIQUE, NOT NULL)
   - `name` (TEXT, NOT NULL)
   - `victory_rule_id` (INTEGER, nullable)
   - `scoring_rule_id` (INTEGER, nullable)
6. **SESSION** - Stocke les sessions de sport
   - `id` (TEXT, PK)
   - `sport_id` (INTEGER, FK vers SPORT, NOT NULL)
   - `point_id` (TEXT, nullable)
   - `state` (TEXT, NOT NULL) - Valeurs: ‘ACTIVE’, ‘TERMINATED’

    - `created_at` (TEXT, NOT NULL)
    - `ended_at` (TEXT, nullable)
    - `winner_participant_id` (TEXT, nullable)
7. **SESSION_PARTICIPANT** - Table de jointure (ManyToMany)
    - `session_id` (TEXT, FK vers SESSION, PK)
    - `participant_id` (TEXT, PK)
    - `participant_type` (TEXT, PK) - Valeurs: ‘PLAYER’, ‘TEAM’
    - `participant_name` (TEXT, NOT NULL)
8. **METRIC_VALUE** - Stocke les métriques des sessions
    - `id` (INTEGER, PK, AUTOINCREMENT)
    - `session_id` (TEXT, FK vers SESSION, NOT NULL)
    - `participant_id` (TEXT, NOT NULL)
    - `metric_type` (TEXT, NOT NULL) - Valeurs: ‘GOALS’, ‘POINTS’, ‘TIME_SECONDS’, ‘REPS’, ‘CUSTOM’
    - `value` (REAL, NOT NULL)
    - `context` (TEXT, nullable)

## Relations

- **EQUIPE** 1—N **JOUEUR** (OneToMany)
- **EQUIPE** 1—0..1 **ARENE** (OneToMany, nullable)
- **ARENE** N—N **SportType** (ManyToMany via ARENE_SPORT)
- **SPORT** 1—N **SESSION** (OneToMany)
- **SESSION** N—N **Participant** (ManyToMany via SESSION_PARTICIPANT)
- **SESSION** 1—N **METRIC_VALUE** (OneToMany)

## Utilisation

### Accéder à la base de données

```
sqlite3 sportsin.db
```

### Commandes SQLite utiles

```
-- Voir toutes les tables
.tables

-- Voir le schéma d'une table
.schema nom_table

-- Voir le schéma complet
.schema

-- Quitter
.quit
```

### Exemples de requêtes

```sql
-- Lister toutes les équipes
SELECT * FROM equipe;

-- Lister tous les joueurs avec leur équipe
SELECT j.id, j.pseudo, e.nom AS equipe
FROM joueur j
LEFT JOIN equipe e ON j.equipe_id = e.id;

-- Lister toutes les arènes avec leur équipe contrôlante
SELECT a.id, a.nom, a.latitude, a.longitude, e.nom AS equipe_controle
FROM arene a
LEFT JOIN equipe e ON a.equipe_controle = e.id;

-- Lister toutes les sessions actives
SELECT s.id, sp.name AS sport, s.state, s.created_at
FROM session s
JOIN sport sp ON s.sport_id = sp.id
WHERE s.state = 'ACTIVE';
```

## Fichiers

- `app/src/main/resources/schema.sql` - Script SQL de création du schéma
- `create_database.sh` - Script shell pour créer la base de données
- `sportsin.db` - Fichier de base de données SQLite (créé après exécution du script)

# Feature 6 : Team Progression & Unlockable Perks

## 1. Vue d'ensemble

La Feature 6 ajoute un **systeme de progression par equipe** au projet SportsIn. Chaque equipe accumule de l'XP en jouant des matchs, monte en niveau, et debloque des **perks activables** (bouclier, boost d'influence, multiplicateur XP) qui modifient le gameplay.

### Objectifs

- Systeme d'XP et de niveaux (1 a 10)
- Catalogue de perks parametrables (non hardcodes)
- Perks consommables avec duree et cooldown
- Integration dans le calcul d'influence existant
- Respect des principes SOLID et des Design Patterns

---

## 2. Architecture & Design Patterns

### Strategy Pattern

L'interface `PerkEffectStrategy` definit le contrat pour chaque type d'effet. Chaque effet est un `@Component` Spring independant. Ajouter un nouveau perk = **1 classe @Component + 1 INSERT SQL**.

```
PerkEffectStrategy (interface)
├── ShieldEffect         (INFLUENCE_REDUCTION)
├── BoostEffect          (INFLUENCE_BOOST)
└── XpMultiplierEffect   (XP_MULTIPLIER)
```

### Registry Pattern

`PerkEffectRegistry` auto-decouvre tous les beans `PerkEffectStrategy` via injection Spring et les indexe par `effectType`. Resolution en O(1).

### Chain of Responsibility (Modifier Chain)

`InfluenceCalculator` chaine des `InfluenceModifier` tries par ordre de priorite : 1. `RouteInfluenceModifier` (ordre 10) — bonus de route existant 2. `PerkInfluenceModifier` (ordre 20) — bonus/malus des perks actifs

### Principes SOLID

- **Open/Closed** : nouveau perk = 1 composant + 1 ligne SQL, aucun code existant a modifier
- **Single Responsibility** : separation donnees (entites) / logique (strategies) / cycle de vie (services)
- **Dependency Inversion** : tout passe par des interfaces (`PerkEffectStrategy`, `InfluenceModifier`)

---

## 3. Systeme de niveaux

Table XP statique dans `LevelThreshold.java` :

| Niveau | XP requis |
|---|---|
| 1 | 0 |
| 2 | 100 |
| 3 | 300 |
| 4 | 600 |
| 5 | 1 000 |
| 6 | 1 500 |
| 7 | 2 200 |
| 8 | 3 000 |

| Niveau | XP requis |
|---|---|
| 9 | 4 000 |
| 10 | 5 500 |

### Gain d'XP

- **Victoire** : +30 XP
- **Defaite** : +10 XP
- Les multiplicateurs XP (perk `XP_BOOST`) s'appliquent automatiquement

---

## 4. Perks disponibles

| Code | Niveau requis | Duree | Cooldown | Effet |
|---|---|---|---|---|
| `SHIELD_QUARTIER` | 3 | 3 jours | 7 jours | -50% influence adverse sur un point |
| `BOOST_INFLUENCE` | 5 | 2 jours | 5 jours | +25% influence propre sur un point |
| `XP_BOOST` | 2 | 24h | 3 jours | x1.5 XP gagne |

Chaque perk est **parametrable via JSON** (champ `parametersJson`), ce qui permet de modifier les pourcentages sans toucher au code.

---

## 5. Endpoints API REST

`GET /api/teams/{teamId}/progression`

Retourne le niveau, l'XP et les perks debloques d'une equipe.

**Reponse :**

```
{
  "teamId": 1,
  "level": 3,
  "currentXp": 350,
  "xpForNextLevel": 250,
  "unlockedPerks": [
    { "code": "XP_BOOST", "name": "Boost XP", "requiredLevel": 2 },
    { "code": "SHIELD_QUARTIER", "name": "Bouclier de Quartier", "requiredLev
el": 3 }
  ]
}
```

`GET /api/perks`

Retourne le catalogue complet des perks.

`POST /api/teams/{teamId}/perks/activate`

Active un perk pour une equipe sur une cible.

**Body :**

```
{
  "perkCode": "SHIELD_QUARTIER",
  "targetId": "42"
}
```

**Validations :** - Niveau suffisant - Nombre max d'instances non atteint - Cooldown ecoule

`GET /api/teams/{teamId}/perks/active`

Retourne les perks actuellement actifs pour une equipe.

---

## 6. Fichiers crees (18 fichiers)

### Modeles

| Fichier | Description |
|---|---|
| `model/progression/PerkDefinition.java` | Entite JPA — catalogue des perks (code, niveau, duree, cooldown, params JSON) |
| `model/progression/ActivePerk.java` | Entite JPA — instance de perk active (equipe, cible, expiration) |
| `model/progression/LevelThreshold.java` | Classe utilitaire — table XP/niveaux statique |
| `model/progression/PerkContext.java` | Record Java — contexte passe aux strategies |

### Strategies (effets)

| Fichier | Description |
|---|---|
| `model/progression/effects/PerkEffectStrategy.java` | Interface Strategy |
| `model/progression/effects/ShieldEffect.java` | Effet INFLUENCE_REDUCTION (-50% influence adverse) |
| `model/progression/effects/BoostEffect.java` | Effet INFLUENCE_BOOST (+25% influence propre) |
| `model/progression/effects/XpMultiplierEffect.java` | Effet XP_MULTIPLIER (x1.5 XP) |
| `model/progression/effects/Perk` | Registre auto-decouverte des strategies |

| Fichier | Description |
|---|---|
| `EffectRegistry.java` | |

### Services

| Fichier | Description |
|---|---|
| `services/TeamProgressionService.java` | Niveau, perks debloques, catalogue |
| `services/PerkActivationService.java` | Activation/desactivation avec validations |
| `services/XpGrantService.java` | Attribution XP avec multiplicateur |
| `services/InfluenceCalculator.java` | Chaine de calcul des modificateurs |
| `services/InfluenceModifier.java` | Interface pour les modificateurs d'influence |
| `services/RouteInfluenceModifier.java` | Modificateur route (extrait de TerritoryService) |
| `services/PerkInfluenceModifier.java` | Modificateur perks (query perks actifs) |

### Controleur

| Fichier | Description |
|---|---|
| `controller/ProgressionController.java` | 4 endpoints REST (voir section 5) |

### Repositories

| Fichier | Description |
|---|---|
| `repository/PerkDefinitionRepository.java` | CRUD + findByCode, findByRequiredLevel |
| `repository/ActivePerkRepository.java` | CRUD + queries actifs/expires/par equipe |

---

## 7. Fichiers modifies (6 fichiers)

| Fichier | Modification |
|---|---|
| `services/TerritoryService.java` | Delegue le calcul de bonus a `InfluenceCalculator` au lieu du calcul inline |
| `services/SessionService.java` | Appelle `XpGrantService.grantMatchXp()` apres chaque match |
| `resources/schema.sql` | Ajout tables `perk_definition` et `active_perk` + 4 index |
| `resources/data.sql` | Ajout des 3 perks initiaux |
| `test/.../TerritoryServiceTest.` | Constructeur mis a jour (ajout InfluenceCalculator) |

| Fichier | Modification |
|---|---|
| `java` | |
| `test/.../SessionServiceTest.java` | Constructeur mis a jour (ajout XpGrantService) |
| `test/.../MissionServiceTest.java` | Constructeur mis a jour (ajout InfluenceCalculator) |

---

## 8. Tests (34 tests Feature 6)

### LevelThresholdTest — 14 tests

Teste le calcul de niveau a partir de l'XP : - Correspondance XP → niveau (0, 99, 100, 300, 1000, 5500, 999999 XP) - XP restant pour le prochain niveau - Seuil XP requis par niveau (valide et invalide) - Cas limites : XP negatif, niveau max depasse

### ShieldEffectTest — 9 tests

Teste l'effet Shield et ses conditions d'activation : - Type d'effet correct (`INFLUENCE_REDUCTION`) - Calcul de reduction (50%, 100%, 0% base, defaut sans parametre) - Rejet si niveau insuffisant - Acceptation au niveau requis - Rejet si max instances atteint - Rejet pendant le cooldown

### InfluenceCalculatorTest — 4 tests

Teste le chainage des modificateurs d'influence : - Deux modifiers chaines dans l'ordre - Aucun modifier → retourne 0 - Un seul modifier - Valeur accumulee transmise au modifier suivant

### TeamProgressionServiceTest — 7 tests

Teste l'integration equipe + systeme de niveaux : - Niveau d'equipe a differents paliers XP (0, 300, 599, 600) - XP necessaire pour le prochain niveau - Progression monotone (le niveau ne descend jamais)

### Resultat global

```
BUILD SUCCESSFUL
67 tests completed, 0 failures (34 nouveaux + 33 existants adaptes)
```

---

## 9. Schema de base de donnees

### Table `perk_definition`

```
CREATE TABLE IF NOT EXISTS perk_definition (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    code TEXT NOT NULL UNIQUE,
    name TEXT NOT NULL,
    description TEXT,
```

```sql
    effect_type TEXT NOT NULL,
    required_level INTEGER NOT NULL DEFAULT 1,
    duration_seconds INTEGER NOT NULL DEFAULT 86400,
    cooldown_seconds INTEGER NOT NULL DEFAULT 0,
    max_active_instances INTEGER NOT NULL DEFAULT 1,
    stackable INTEGER NOT NULL DEFAULT 0,
    parameters_json TEXT
);
```

### Table `active_perk`

```sql
CREATE TABLE IF NOT EXISTS active_perk (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    team_id INTEGER NOT NULL,
    perk_definition_id INTEGER NOT NULL,
    target_id TEXT,
    activated_at TEXT NOT NULL,
    expires_at TEXT NOT NULL,
    last_used_at TEXT,
    usage_count INTEGER NOT NULL DEFAULT 0,
    FOREIGN KEY (team_id) REFERENCES equipe(id),
    FOREIGN KEY (perk_definition_id) REFERENCES perk_definition(id)
);
```

---

## 10. Diagramme de flux

```
Match termine
     │
     ▼
SessionService.endSession()
     │
     ├─► XpGrantService.grantMatchXp(winner, true)   → +30 XP (× multiplicate
ur)
     ├─► XpGrantService.grantMatchXp(loser, false)    → +10 XP (× multiplicat
eur)
     │
     ▼
Equipe.xp mis a jour en BDD
     │
     ▼
LevelThreshold.levelForXp(xp) → nouveau niveau
     │
     ▼
Nouveaux perks debloques si niveau suffisant
     │
     ▼
POST /api/teams/{id}/perks/activate
     │
     ├─► Validation : niveau, cooldown, max instances
```

```
      ├─► Creation ActivePerk en BDD
      │
      ▼
Calcul d'influence (TerritoryService)
      │
      ├─► InfluenceCalculator.computeTotalModifier()
      │       ├─► RouteInfluenceModifier  (ordre 10)
      │       └─► PerkInfluenceModifier   (ordre 20)
      │
      ▼
Score final = score de base + modificateur total
```

---

## 11. Comment ajouter un nouveau perk

1. **Creer une classe Strategy** :

```java
@Component
public class NouvelEffet implements PerkEffectStrategy {
    @Override
    public String getEffectType() { return "NOUVEL_EFFET"; }

    @Override
    public double computeInfluenceModifier(PerkContext ctx) {
        // logique de l'effet
    }

    @Override
    public boolean canActivate(Equipe team, PerkDefinition def, List<ActivePe
rk> active) {
        // conditions d'activation
    }
}
```

2. **Ajouter une ligne SQL** dans `data.sql` :

```sql
INSERT INTO perk_definition (code, name, description, effect_type, required_l
evel,
    duration_seconds, cooldown_seconds, max_active_instances, stackable, para
meters_json)
VALUES ('NOUVEL_EFFET', 'Mon Nouveau Perk', 'Description...', 'NOUVEL_EFFET',
    4, 172800, 432000, 1, 0, '{"param": 30}');
```

Aucune autre modification necessaire grace au principe **Open/Closed**.

---

## 12. Branche Git

Tout le code se trouve sur la branche `feature/team-progression`.

# Feature 3 : Calcul Automatique de Contrôle de Points et de Zones

Cette fonctionnalité introduit la dimension stratégique territoriale dans l'application SportsIn. Elle permet de gérer dynamiquement la conquête de points sportifs et de zones géographiques en fonction des résultats des sessions de sport.

## 1. Objectifs

- **Contrôle de Point** : Une équipe prend le contrôle d'un point sportif (ex: un City Stade) lorsqu'elle gagne une session sur ce point.
- **Génération de Zones** : Regroupement automatique des points proches géographiquement pour former des "Zones".
- **Contrôle de Zone** : Une équipe prend le contrôle d'une zone entière si elle possède au moins **3 points** dans cette zone.

## 2. Fonctionnement Technique

### A. Génération Automatique des Zones (`ZoneGeneratorService`)

Au démarrage (ou sur demande), l'application analyse tous les points sportifs disponibles. * **Algorithme** : Clustering géographique simple. * **Critères** : * **Proximité** : Les points doivent être dans un rayon défini (ex: 2 km). * **Densité** : Une zone n'est créée que si elle contient un minimum de points (ex: 3 points). * **Résultat** : Création d'objets `Zone` persistés en base, contenant la liste des points associés.

### B. Mécanique de Conquête (`TerritoryService`)

À la fin de chaque session de sport (`SessionService`), si un vainqueur est désigné : 1. **Mise à jour du Point** : Le point sportif change de propriétaire (`controllingTeamId`). 2. **Calcul d'Impact Zone** : Le système vérifie toutes les zones contenant ce point. 3. **Application de la Règle des 3 Points** : * Le système compte les points contrôlés par chaque équipe dans la zone. * **Conquête** : Si une équipe atteint **>= 3 points**, elle devient propriétaire de la zone. * **Perte** : Si le propriétaire actuel passe **< 3 points**, la zone redevient neutre (ou change de main si une autre équipe a >= 3 points).

## 3. Architecture et Composants Clés

| Composant | Rôle |
|---|---|
| **`Zone`** (Model) | Représente un ensemble de points. Contient la logique `updateZoneControl()` pour vérifier la règle des 3 points. |
| **`GeoUtils`** (Utils) | Fournit le calcul de distance GPS (formule de Haversine). |
| **`ZoneGeneratorService`** | Service responsable de créer les zones à partir des coordonnées GPS des points. |

| Composant | Rôle |
| --- | --- |
| **`TerritoryService`** | Service centralisant la logique de mise à jour des territoires (Points & Zones). |
| **`SessionService`** | Chef d'orchestre qui appelle `TerritoryService` une fois qu'une session est validée. |

## 4. Exemple de Scénario (Logique)

1. **Initialisation** : Le système détecte 3 stades proches à Paris (Châtelet, Louvre, Notre-Dame) et crée la "Zone Paris".
2. **Match 1** : L'équipe "Les Requins" gagne à Châtelet.
    - *État* : Châtelet = Requins. Zone Paris = Neutre (1/3).
3. **Match 2** : "Les Requins" gagnent au Louvre.
    - *État* : Louvre = Requins. Zone Paris = Neutre (2/3).
4. **Match 3** : "Les Requins" gagnent à Notre-Dame.
    - *État* : Notre-Dame = Requins. **Zone Paris = CONQUISE par Les Requins** (3/3).

## 5. Tests

Les tests unitaires couvrent l'ensemble de la logique : * `ZoneGeneratorServiceTest` : Vérifie que les points proches sont bien groupés et les points lointains exclus. * `TerritoryServiceTest` : Vérifie la bascule de propriété des points et des zones (conquête et perte). * `SessionServiceTest` : Vérifie l'intégration complète depuis la fin d'une session.

# Feature 4 : Routes sportives & bonus de combo

## Objectif

Implémenter un système de "routes sportives" (graphe de points) permettant aux équipes de débloquer des bonus lorsqu'elles contrôlent une suite de points consécutifs.

## Statut

**OPÉRATIONNELLE** - La feature est entièrement implémentée, testée et intégrée.

## Architecture & Implémentation

### 1. Modèle de Données

- **Route** (`org.SportsIn.model.territory.Route`) : Séquence ordonnée de `PointSportif`.
- **RouteBonus** (`org.SportsIn.model.territory.RouteBonus`) : Objet représentant un bonus actif (ex: +10% de score).

- **RouteRepository** (`org.SportsIn.model.territory.RouteRepository`) : Interface et implémentation (`InMemoryRouteRepository`) pour la persistance des routes.

### 2. Services Métier

- **RouteService** :
  - Algorithme de détection de chaînes consécutives (`getMaxConsecutivePoints`).
  - Calcul des bonus actifs (`calculateBonuses`).
- **RouteGeneratorService** :
  - Algorithme “Greedy Nearest Neighbor” pour générer automatiquement des routes à partir des points géographiques.
- **TerritoryService** :
  - Orchestre la conquête.
  - `initializeRoutesAutomatically` : Initialise le graphe au démarrage.
  - `getScoreBonusForTeamOnPoint` : Vérifie si un bonus s’applique lors d’une session.
  - `updateTerritoryControl` : Met à jour le propriétaire du point, vérifie les zones et les routes.
- **SessionService** :
  - Intègre le bonus de route dans le flux de fin de session (`processSessionCompletion`).
  - Loggue l’application du bonus (prêt pour modification du score).

### 3. API & Configuration

- **RouteController** (`/api/routes`) :
  - Endpoint `GET` pour exposer les routes au Frontend (visualisation sur carte).
- **RouteInitializer** :
  - `CommandLineRunner` qui génère automatiquement les routes au démarrage de l’application (distance max 2km, min 3 points).

### 4. Tests

- `RouteServiceTest` : Algorithmes de graphe.
- `RouteGeneratorServiceTest` : Génération géographique.
- `TerritoryServiceTest` : Intégration complète (Repository, Services).
- `SessionServiceTest` : Vérification de la non-régression sur la fin de session.

## Flux Fonctionnel

1. **Démarrage** : `RouteInitializer` appelle le générateur -> Les routes sont créées et stockées dans `RouteRepository`.
2. **Jeu** : Une équipe gagne une session sur un point.
3. **Calcul** : `SessionService` demande à `TerritoryService` si un bonus s’applique.

4. **Bonus** : Si l'équipe contrôle une suite de points (>=3) sur une route incluant le point actuel, un bonus est appliqué.
5. **Conquête** : Le point change de main, `TerritoryService` recalcule les zones et les routes pour la prochaine fois.

# Feature 5 : Systeme de Missions Dynamiques

## 1. Vue d'ensemble

La Feature 5 ajoute un **systeme de missions dynamiques** au projet SportsIn. Chaque equipe recoit automatiquement des missions basees sur l'etat du jeu (arenes controlees, sports pratiques, routes adverses). Les missions sont generees quotidiennement, evaluees en continu, et recompensent les equipes avec des points et de l'XP.

### Objectifs

- Generation automatique de missions contextuelles (max 3 par equipe)
- Trois types de missions : Reconquete, Diversite de sport, Rupture de route
- Evaluation periodique via scheduler Spring
- Systeme de recompenses (points + XP)
- Affichage des missions sur la carte interactive (marqueurs dores)

---

## 2. Architecture & Design Patterns

### Strategy Pattern

`MissionEvaluationService` utilise une evaluation specifique par type de mission. Chaque type (`RECAPTURE_RECENT_LOSS`, `DIVERSITY_SPORT`, `BREAK_ROUTE`) possede sa propre logique de validation.

### Repository Pattern

`MissionRepository` abstrait l'acces aux donnees avec des requetes optimisees et des index performants.

### DTO Pattern

Separation API via `MissionSummaryDTO` (liste) et `MissionDetailDTO` (detail complet avec payload JSON parse).

### Payload Pattern

Chaque mission stocke un `payloadJson` flexible contenant les parametres specifiques (arenaId, sportCode, etc.), permettant d'ajouter de nouveaux types sans modifier le schema.

### Scheduled Tasks

Spring `@Scheduled` pour la gestion automatique du cycle de vie des missions (generation, evaluation, expiration).

---

## 3. Types de Missions

| Type | Objectif | Duree | Points | XP | Priorite |
|---|---|---|---|---|---|
| `RECAPTURE_RECENT_LOSS` | Reconquerir une arene controlee par l'adversaire | 3 jours | 50 | 30 | HIGH |
| `DIVERSITY_SPORT` | Pratiquer un sport non joue depuis 14 jours sur une arene | 7 jours | 30 | 20 | LOW |
| `BREAK_ROUTE` | Capturer une 2e arene adverse pour briser une route | 5 jours | 75 | 50 | MEDIUM |

### Regles de generation (R1, R2, R3)

- **R1 (Reconquete)** : Declenchee quand une arene est controlee par un adversaire. Payload : `{arenaId, arenaName, windowDays: 7}`
- **R2 (Diversite)** : Declenchee quand un sport disponible sur une arene n'a pas ete joue depuis 14 jours. Payload : `{arenaId, arenaName, sportCode, windowDays: 14}`
- **R3 (Rupture)** : Declenchee quand un adversaire controle 2+ arenes. Payload : `{arenaId, arenaName, adversaryTeamId, minCount: 1}`

---

## 4. Scheduler

`MissionScheduler` gere le cycle de vie automatique :

| Tache | Frequence | Action |
|---|---|---|
| Expiration & Evaluation | Toutes les 10 minutes | Expire les missions depassees, evalue les missions actives |
| Generation quotidienne | Chaque jour a 06:00 (Europe/Paris) | Genere de nouvelles missions pour toutes les equipes |

Configuration : desactivable via `mission.scheduler.enabled=false` dans `application.properties`.

---

## 5. Endpoints API REST

`GET /api/teams/{teamId}/missions`

Liste les missions d'une equipe (filtre optionnel par statut).

**Parametre query :** `?status=ACTIVE`

**Reponse :** Liste de `MissionSummaryDTO`

```
[
  {
    "id": 1,
    "type": "RECAPTURE_RECENT_LOSS",
    "status": "ACTIVE",
    "title": "Reconquerir Chatelet",
    "priority": "HIGH",
    "endsAt": "2025-01-15T06:00:00",
    "rewardTeamPoints": 50,
    "progressCurrent": 0,
    "progressTarget": 1
  }
]
```

`GET /api/missions/{missionId}`

Retourne le detail complet d'une mission (`MissionDetailDTO`).

`POST /api/teams/{teamId}/missions/generate`

Force la generation de missions (debug/test).

`POST /api/missions/{missionId}/refresh`

Force l'evaluation d'une mission (debug/test).

**Tri des resultats :** - Missions ACTIVE : triees par `endsAt` ASC (urgentes en premier) - Missions terminees : triees par `completedAt` DESC (recentes en premier)

---

## 6. Fichiers crees (15 fichiers)

### Modeles

| Fichier | Description |
|---|---|
| `model/mission/Mission.java` | Entite JPA — mission avec cycle de vie complet (statut, progression, payload JSON) |
| `model/mission/MissionType.java` | Enum — 3 types : `RECAPTURE_RECENT_LOSS`, `BREAK_ROUTE`, `DIVERSITY_SPORT` |

| Fichier | Description |
| --- | --- |
| `model/mission/MissionStatus.java` | Enum — 4 statuts : `ACTIVE`, `SUCCESS`, `FAILED`, `EXPIRED` |
| `model/mission/MissionPriority.java` | Enum — 3 niveaux : `LOW`, `MEDIUM`, `HIGH` |

## DTOs

| Fichier | Description |
| --- | --- |
| `dto/MissionSummaryDTO.java` | Donnees minimales pour les listes (id, type, statut, progression, recompense) |
| `dto/MissionDetailDTO.java` | Donnees completes avec payload JSON parse pour le frontend |

## Services

| Fichier | Description |
| --- | --- |
| `services/MissionGenerationService.java` | Generation dynamique des 3 types de missions (R1, R2, R3) avec deduplication |
| `services/MissionEvaluationService.java` | Evaluation des conditions de reussite, attribution des recompenses, gestion expiration |

## Scheduler

| Fichier | Description |
| --- | --- |
| `scheduler/MissionScheduler.java` | Taches planifiees : generation quotidienne + evaluation toutes les 10 min |

## Controleur

| Fichier | Description |
| --- | --- |
| `controller/MissionController.java` | 4 endpoints REST (liste, detail, generation, refresh) |

## Repository

| Fichier | Description |
| --- | --- |
| `repository/MissionRepository.java` | CRUD + requetes optimisees (par equipe, statut, expiration) |

## Tests

| Fichier | Description |
| --- | --- |
| `test/.../MissionServiceTest.java` | Tests unitaires complets avec stubs in-memory |

## Frontend

| Fichier | Description |
| --- | --- |

| Fichier | Description |
|---|---|
| `frontend/src/api/api.js` | Facade API missions (getByTeam, getById, generate, refresh) |
| `frontend/src/pages/MapPage.jsx` | Affichage des missions sur la carte (marqueurs dores, popups, countdown) |

---

## 7. Fichiers modifies

| Fichier | Modification |
|---|---|
| `resources/schema.sql` | Ajout table `mission` + 3 index de performance |

---

## 8. Tests

### MissionServiceTest — 9 tests

**Tests de generation :** - `testGenerateDoesNotExceedMaxActive()` — Respecte la limite de 3 missions actives - `testGenerateRecaptureMission()` — Creation correcte d'une mission RECAPTURE_RECENT_LOSS - `testGenerateDiversityMission()` — Creation correcte d'une mission DIVERSITY_SPORT - `testGenerateBreakRouteMissionFromArenas()` — Creation correcte d'une mission BREAK_ROUTE

**Tests d'evaluation :** - `testMissionExpires()` — Logique d'expiration (statut → EXPIRED) - `testEvaluateRecaptureSuccess()` — Detection du controle d'arene - `testEvaluateRecaptureStillActive()` — Condition non remplie → reste ACTIVE - `testEvaluateBreakRouteSuccess()` — Capture de la 2e arene adverse - `testEvaluateAllActiveMissions()` — Evaluation par lot

**Stubs in-memory :** - `InMemoryMissionRepository` - `InMemoryEquipeRepository` - `InMemoryAreneRepository` - `InMemorySessionRepository`

---

## 9. Schema de base de donnees

### Table `mission`

```sql
CREATE TABLE IF NOT EXISTS mission (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    team_id INTEGER NOT NULL,
    type TEXT NOT NULL CHECK (type IN ('RECAPTURE_RECENT_LOSS', 'BREAK_ROUTE',
 'DIVERSITY_SPORT')),
    status TEXT NOT NULL DEFAULT 'ACTIVE',
    title TEXT NOT NULL,
    description TEXT,
    priority TEXT NOT NULL DEFAULT 'MEDIUM',
    reward_team_points INTEGER NOT NULL DEFAULT 0,
```

```sql
    reward_team_xp INTEGER NOT NULL DEFAULT 0,
    created_at TEXT NOT NULL,
    starts_at TEXT NOT NULL,
    ends_at TEXT NOT NULL,
    completed_at TEXT,
    payload_json TEXT,
    progress_current INTEGER NOT NULL DEFAULT 0,
    progress_target INTEGER NOT NULL DEFAULT 1,
    last_evaluated_at TEXT,
    FOREIGN KEY (team_id) REFERENCES equipe(id) ON DELETE CASCADE
);
```

### Index de performance

```sql
CREATE INDEX idx_mission_team_id ON mission(team_id);
CREATE INDEX idx_mission_status ON mission(status);
CREATE INDEX idx_mission_team_status ON mission(team_id, status);
```

---

## 10. Diagramme de flux

```
=== GENERATION (quotidien 06:00) ===

MissionScheduler.generateMissionsForAllTeams()
     │
     ▼
Pour chaque equipe :
     │
     ▼
MissionGenerationService.generateForTeam(teamId)
     │
     ├─► Verifier : missions actives < 3 ?
     │
     ├─► R1 : Arene adverse ? → RECAPTURE_RECENT_LOSS (50 pts, 30 XP)
     ├─► R2 : Sport non joue depuis 14j ? → DIVERSITY_SPORT (30 pts, 20 XP)
     ├─► R3 : Adversaire controle 2+ arenes ? → BREAK_ROUTE (75 pts, 50 XP)
     │
     ├─► Deduplication via payloadKey()
     │
     ▼
Sauvegarde en BDD (MissionRepository)


=== EVALUATION (toutes les 10 min) ===

MissionScheduler.expireAndEvaluate()
     │
     ├─► Expiration : missions depassees → statut EXPIRED
     │
     ▼
```

```
MissionEvaluationService.evaluateAllActiveMissions()
      │
      ├─► RECAPTURE : equipe controle l'arene ? → SUCCESS
      ├─► DIVERSITY : session du sport sur l'arene apres debut mission ? → SUC
CESS
      ├─► BREAK_ROUTE : equipe controle l'arene cible ? → SUCCESS
      │
      ▼
Si SUCCESS :
      ├─► Ajout rewardTeamPoints a l'equipe
      ├─► Ajout rewardTeamXp a l'equipe
      ├─► Statut → SUCCESS, progression → 1/1
      │
      ▼
Mise a jour en BDD


=== AFFICHAGE FRONTEND ===

MapPage.jsx
      │
      ├─► Fetch missions actives pour l'equipe courante
      ├─► Construction map missionsByArena
      │
      ▼
Carte interactive :
      ├─► Marqueurs dores sur les arenes avec missions
      ├─► Popup : type (Reconquete / Rupture / Diversite)
      ├─► Barre de progression
      ├─► Recompense en points
      └─► Compte a rebours (jours / heures / minutes)
```

---

# 11. Integration Frontend

## Marqueurs de mission

Les arenes associees a une mission active sont identifiees par un **marqueur dore** special sur la carte Leaflet.

## Labels francais

| Type | Label affiche |
|---|---|
| `RECAPTURE_RECENT_LOSS` | Reconquete |
| `BREAK_ROUTE` | Rupture de route |
| `DIVERSITY_SPORT` | Diversite sport |

### Popup de mission

Chaque marqueur de mission affiche : - Le type de mission avec son label - Le titre et la description - Une barre de progression (`progressCurrent / progressTarget`) - Les points de recompense - Le temps restant (formate en jours/heures/minutes)

---

## 12. Branche Git

Commits associes : - `8b3f00d feat(feature5): ajout systeme missions dynamiques (entity, service, controller, scheduler)` - `298a025 feat(feature5): ajout systeme de missions dynamiques (entity, service, controller, scheduler)` - `2323296 mini reorganisation - ajouter recap F5`

# □ Mécaniques de Jeu - InSport

Ce document détaille les règles et les mécaniques de jeu d'InSport.

## □ Territoire & Points d'Intérêt

Le jeu se déroule sur une carte de l'Île-de-France.

### □ Points d'Intérêt (POI)

Chaque point représente un lieu sportif réel (parc, city-stade, gymnase, piste, salle…). * **Sports disponibles** : Chaque point peut accueillir plusieurs sports (Foot, Basket, Musculation, Course…). * **Influence** : Les équipes gagnent de l'influence sur un point en réalisant des sessions sportives. * **Contrôle** : Lorsqu'une équipe domine un point de manière suffisante, elle le contrôle officiellement.

### □ Zones

Les points sont regroupés en zones géographiques (quartiers, parcs, secteurs). * **Domination de Zone** : Si une équipe contrôle 3 points dans une zone, elle domine la zone entière.

## □ Routes Sportives (Innovation)

Certains points sont reliés entre eux pour former des chemins stratégiques (via un quartier, un axe ou une ligne de RER).

- **Contrôle de Route** : Si une équipe contrôle plusieurs points consécutifs sur une route, elle débloque un **bonus spécial**.
- **Bonus possibles** :
  - Avantage dans un sport spécifique.
  - Protection temporaire contre les attaques adverses.
  - Accès à des missions avancées.

Ce système pousse à une conquête organisée : contrôle de zones, coupures de routes adverses, stratégies d’expansion.

## □ Sessions Sportives & Conquête

1. **Regroupement** : Les joueurs se regroupent en équipes.
2. **Déplacement** : Ils se rendent physiquement sur un point de la carte.
3. **Session** : Ils organisent une session sportive selon un sport disponible sur ce point.
4. **Soumission** : Le résultat de la session (score, performance, temps, etc.) est soumis via le front-end.
5. **Résolution** : Le back-end Java applique les règles du sport pour déterminer le vainqueur et met à jour l’influence.

## □ Missions Dynamiques

Le jeu propose des missions pour encourager l’activité physique : * Missions à durée limitée. * Missions liées à la conquête de routes spécifiques. * Missions de défense de territoire.

## □ Diagramme des Cas d’Utilisation

Ce diagramme illustre les actions possibles pour les joueurs et les administrateurs.

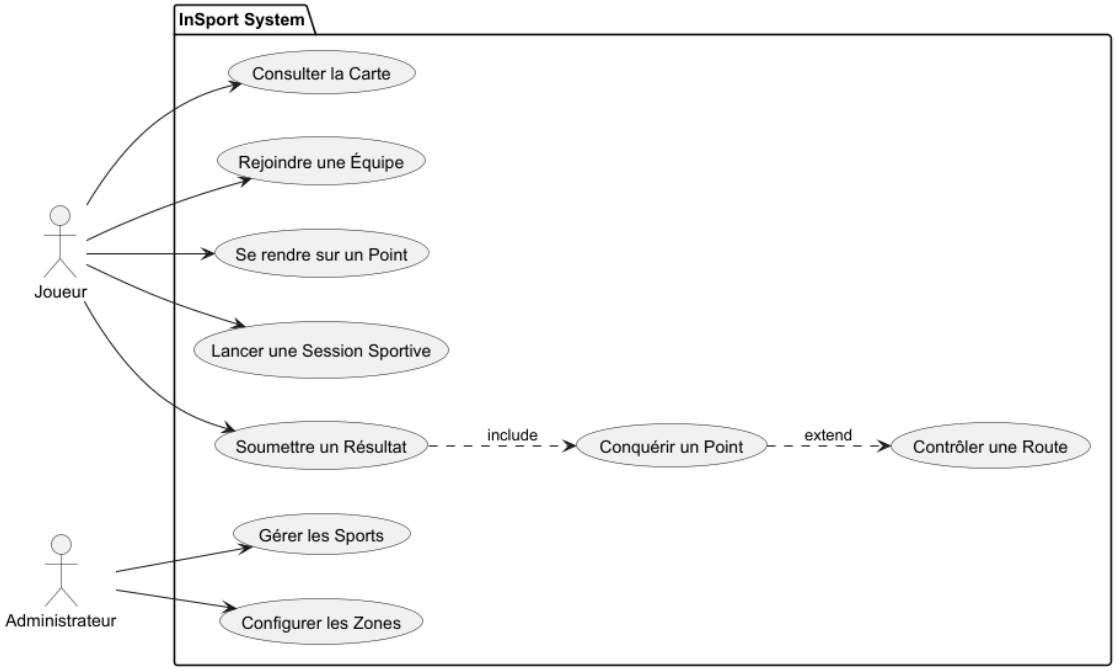


*Diagramme des Cas d’Utilisation*

# □ Plan de Livraisons - InSport

Ce document détaille le plan de développement du projet InSport, découpé en 3 livraisons majeures, chacune apportant 2 fonctionnalités clés.

L'objectif est de construire progressivement la complexité du jeu, en partant du moteur de règles sportives pour arriver à la couche stratégique (routes, missions, progression).

---

## □ Livraison 1 : Socle du Jeu & Règles Sportives

Cette première livraison pose les fondations du moteur de jeu. Elle permet aux joueurs de réaliser des sessions sportives et au système de déterminer les vainqueurs selon des règles configurables.

### ✨Feature 1 : Moteur de règles de victoire multi-sports configurable

**Concept :** Chaque sport possède ses propres conditions de victoire et de scoring, définies côté Backend et exposées via l'API. Le Frontend est agnostique et se contente d'afficher les règles et de soumettre les résultats bruts.

**Exemples de règles configurables :** * **Foot :** "Gagne l'équipe qui marque le plus de buts". * **Musculation :** "Gagne l'équipe avec le meilleur total pondéré sur 3 exercices". * **Course :** "Gagne le meilleur temps moyen sur une distance donnée". * **Basket 3x3 :** "Premier à 21 points ou meilleur score après 10 min".

**Complexité Technique :** * Implémentation d'un **Pattern Strategy** ou d'un moteur de règles pour interpréter les conditions de victoire. * Séparation stricte entre les données brutes (temps, score) et la logique de validation.

### ✨Feature 2 : Gestion des sessions de défi & Validation

**Concept :** Permettre aux équipes de s'affronter via un workflow complet de gestion de session.

**Workflow :** 1. **Création :** Une équipe lance un défi sur un Point (Lieu + Sport + Créneau). 2. **Inscription :** Une équipe adverse relève le défi. 3. **Réalisation :** Le match a lieu physiquement. 4. **Soumission :** Une équipe saisit le résultat. 5. **Validation Croisée :** L'autre équipe doit confirmer le score. 6. **Conflit :** En cas de désaccord, la session passe en statut "Conflit" pour arbitrage admin.

**Complexité Technique :** * Machine à états (State Machine) pour gérer le cycle de vie d'une session. * Gestion de la concurrence et des délais de validation.

---

## □ Livraison 2 : Territoire, Zones & Routes (Innovation)

Cette livraison introduit la dimension stratégique et territoriale, transformant l'application en un véritable jeu de conquête.

### ✨Feature 3 : Calcul de contrôle de Points & Zones

**Concept :** Le Backend calcule en continu l'influence des équipes sur la carte en fonction de l'historique des sessions.

**Mécaniques :** * **Influence par Point :** Basée sur les victoires récentes (avec décroissance temporelle possible). * **Domination de Zone :** Si une équipe contrôle X points dans une zone (quartier/parc), elle contrôle la zone entière. * **Feedback Joueur :** "Vous contrôlez 4/7 points de la zone La Défense".

**Complexité Technique :** * Algorithmes d'agrégation de scores sur des fenêtres temporelles. * Gestion des transitions d'état (Point Neutre -> Contesté -> Contrôlé).

### ✨Feature 4 : Routes Sportives & Bonus de Combo (Innovation Majeure)

**Concept :** Les points sont reliés entre eux pour former un **Graphe**. Contrôler une suite de points connectés (une "Route") octroie des bonus stratégiques.

**Exemple :** * **Route "RER B Sud" :** Relie les points A -> B -> C -> D -> E. * **Combo :** Si l'équipe Rouge contrôle A, B et C (3 points consécutifs), elle active un bonus (ex: +10% score running). * **Stratégie :** Les adversaires peuvent tenter de prendre le point B pour "couper" la route et désactiver le bonus.

**Complexité Technique :** * Modélisation de graphe (Noeuds = Points, Arêtes = Routes). * Algorithmes de détection de sous-graphes connectés (chaînes consécutives) appartenant à une même équipe.

---

## □ Livraison 3 : Engagement & Personnalisation

Cette dernière livraison vise à fidéliser les joueurs via des objectifs dynamiques et un système de progression RPG.

### ✨Feature 5 : Missions Dynamiques

**Concept :** Le système génère des quêtes contextuelles pour orienter l'action des joueurs.

**Types de Missions :** * **Reconquête :** "Reprendre le point X perdu il y a 2 jours". * **Sabotage :** "Briser la route de l'équipe Verte sur la ligne RER B". * **Diversité :** "Faire une session de Musculation dans une zone où ce sport est peu pratiqué".

**Complexité Technique :** * Moteur de génération procédurale de missions basé sur l'état actuel du monde (World State). * Suivi de la complétion des objectifs en temps réel.

### ✨ Feature 6 : Progression d'Équipe & Perks

**Concept :** Les équipes gagnent de l'expérience (XP) et débloquent des avantages passifs ou actifs (Perks).

**Exemples de Perks :** * **Bouclier :** Protéger un point clé contre les attaques pendant 24h. * **Spécialiste :** Bonus de points sur un sport spécifique. * **Résistance :** Réduction de la perte d'influence quotidienne.

**Complexité Technique :** * Système de calcul d'XP multi-sources (Matchs, Missions, Routes). * Gestion des effets actifs/passifs et de leur impact sur les calculs du moteur de jeu.

---

## □ Résumé

| Livraison | Features Clés | Valeur Ajoutée |
|---|---|---|
| **L1** | Règles Multi-sports, Workflow Sessions | **Jouabilité de base** (Faire du sport et compter les points) |
| **L2** | Contrôle Territoire, **Routes Sportives** | **Stratégie & Innovation** (Jeu de conquête, Graphes) |
| **L3** | Missions, Progression RPG | **Engagement & Rétention** (Objectifs à long terme) |